

02 - Development Environment Setup

Everything you must install on your laptop, with the exact versions this project needs.

What is this?

A checklist of programs to install before you can run Hardware ERP. Each entry tells you why it is needed, how to check whether you already have it, how to install it, and how to prove it worked.

IMPORTANT

The versions below are read from the project files
They are not general advice. backend/pom.xml sets Java 21 and Spring Boot 3.4.2. frontend/package.json sets Vite 6 and TypeScript 5.7. Installing a different major version will cause errors that look unrelated to the version.

The shape of the work

BEFORE	AFTER
nothing installed	three programs running
[1] Install Java 21 -----+ [2] Install Maven -----+----> backend can build [3] Install Docker -----+----> PostgreSQL can run [4] Install Node 20+ -----+----> frontend can build [5] Install Git -----+----> you can clone the code [6] Install Postman -----+----> you can test the APIs [7] Install an IDE -----+----> you can read and edit	

Version summary

Tool	Required version	Where this comes from
JDK (Java)	21	pom.xml property java.version
Maven	3.9+, or use the bundled wrapper	Spring Boot 3.4 requires 3.6.3+
Node.js	20.19+ or 22.12+	Vite 6 requirement
npm	10+	Ships with Node 20 and 22
Docker	any current version	Runs PostgreSQL 16 and Testcontainers
PostgreSQL	16	docker-compose.yml image tag

Tool	Required version	Where this comes from
Git	any current version	Cloning the repository
Postman	any current version	API testing
IDE	IntelliJ IDEA or VS Code	Preference
Browser	Chrome, Edge or Firefox	Running the frontend

1. JDK 21

WHY

Why

The backend is written in Java. The JDK is the kit that compiles and runs it. `pom.xml` declares `<java.version>21</java.version>`, so Java 17 will not compile this code and Java 25 is untested here.

RUN THIS

Check whether you already have it

```
java -version
```

WHAT YOU SHOULD SEE

Good output

```
openjdk version "21.0.10" 2026-01-20
```

Any `21.x.x` is fine. If you see 17 or 8, or "command not found", install it.

Install: download Eclipse Temurin 21 (LTS) from adoptium.net, or:

- Windows: `winget install EclipseAdoptium.Temurin.21.JDK`
- macOS: `brew install --cask temurin@21`
- Ubuntu/Debian: `sudo apt install openjdk-21-jdk`

VERIFY

Prove it

Close and reopen your terminal, then run `java -version` again. It must print 21. If it still prints an old version, your `JAVA_HOME` or `PATH` still points at the old JDK.

2. Maven

WHY

Why

Maven downloads the libraries the backend depends on, compiles the code, runs the tests and packages everything into one runnable JAR file.

IMPORTANT

You probably do not need to install Maven

The project includes the **Maven Wrapper**. Use `./mvnw` on macOS and Linux, or `mvnw.cmd` on Windows, and it downloads the correct Maven version by itself.

This is the recommended way, because everyone then uses the same version.

RUN THIS

Check

```
mvn -version
```

WHAT YOU SHOULD SEE

Good output

```
Apache Maven 3.9.x
```

Install if you want it globally:

- Windows: `winget install Apache.Maven`
- macOS: `brew install maven`
- Ubuntu/Debian: `sudo apt install maven`

3. Docker

WHY

Why

Two reasons. First, it runs PostgreSQL without you installing a database server directly on your laptop. Second, the integration tests use

Testcontainers, which starts a throwaway PostgreSQL container for the test run. Without Docker those tests cannot run at all.

RUN THIS

Check

```
docker --version
```

```
docker compose version
```

WHAT YOU SHOULD SEE

Good output

```
Docker version 27.x.x
```

```
Docker Compose version v2.x.x
```

Install: Docker Desktop from docker.com/products/docker-desktop for Windows and macOS. On Ubuntu, follow the official docs for Docker Engine plus the Compose plugin.

IF IT FAILS

"Cannot connect to the Docker daemon"

Docker is installed but not started. Open Docker Desktop and wait for the whale icon to stop animating. On Linux: `sudo systemctl start docker`.

4. Node.js and npm

WHY

Why

Node runs the frontend build tools. npm downloads the frontend libraries.

Neither is used at runtime by the finished frontend - the browser runs that - but both are needed to build it.

RUN THIS

Check

```
node --version
npm --version
```

WHAT YOU SHOULD SEE

Good output

```
v22.x.x      (or v20.19 or higher)
10.x.x
```

IMPORTANT

Vite 6 will refuse to start on Node 18

The error message mentions `crypto.hash is not a function` and does not mention Node at all, so this one wastes a lot of time if you miss it.

Install: download the LTS build from nodejs.org, or:

- Windows: `winget install OpenJS.NodeJS.LTS`
- macOS: `brew install node@22`
- Ubuntu: use [nodesource](https://nodesource.com), or `nvm install 22`

5. PostgreSQL

WHY

Why

The database. See document 03 for what a database actually is and how to use pgAdmin.

IMPORTANT

You do not need to install PostgreSQL separately
docker-compose.yml in the project root starts PostgreSQL 16 for you, already
configured with the right database name, user and password. This is the
recommended path. Document 03 covers both options.

RUN THIS

Start it

```
cd hardware-erp
docker compose up -d
```

WHAT YOU SHOULD SEE

Expected

```
[+] Running 2/2
   Network hardware-erp_default      Created
   Container hardware-erp-postgres   Healthy
```

6. Git

WHY

Why

Downloads the project and records your changes so you can undo them.

RUN THIS

Check

```
git --version
```

Install: `git-scm.com/downloads`, or `winget install Git.Git`,
`brew install git`, `sudo apt install git`.

7. Postman

WHY

Why

Sends API requests without the frontend, so you can tell whether a fault is in
the backend or the screen. Document 09 covers it fully, and the project ships
a ready-made collection.

Install: `postman.com/downloads`.

8. IDE

WHY

Why

An editor that understands the code - jump to a definition, autocomplete, show errors before you compile.

IDE	Good for	Notes
IntelliJ IDEA	Backend (Java)	Community Edition is free and sufficient
VS Code	Frontend (React)	Free. Add the "ESLint" and "Tailwind CSS IntelliSense" extensions

Both open the project directly. In IntelliJ use **File > Open** and select the backend folder so it detects the `pom.xml`.

Putting it together

Once everything above is installed, this is the whole startup sequence:

RUN THIS

Terminal 1 - database

```
cd hardware-erp
docker compose up -d
```

RUN THIS

Terminal 2 - backend

```
cd hardware-erp/backend
./mvnw spring-boot:run
```

RUN THIS

Terminal 3 - frontend

```
cd hardware-erp/frontend
npm install
npm run dev
```

WHAT YOU SHOULD SEE

All three healthy

- Database: `docker compose ps` shows the container as healthy
- Backend: the console ends with `Started HardwareErpApplication in X seconds`
- Frontend: the console prints `Local: http://localhost:5173/`
- Browser: opening `http://localhost:5173` shows the sign-in screen

Verification checklist

VERIFY

Run each command and confirm the output

```
java -version          -> openjdk version "21..."
./mvnw -version        -> Apache Maven 3.9...
node --version         -> v20.19+ or v22...
npm --version          -> 10...
docker --version       -> Docker version 27...
git --version          -> git version 2...
docker compose ps      -> hardware-erp-postgres ... healthy
curl localhost:8080/api/actuator/health -> {"status":"UP"}
```

If all eight pass, your laptop is ready. If any fails, document 17 has the fix.